

05__backtest

June 29, 2026

1 05 — Backtest: does copying the sharps actually pay?

The moment of truth. Everything before this was plumbing. This notebook tests the *premise*: if you had copied the roster sharps' bets — at their entry price plus slippage, held to resolution — would you have made money? And does requiring more **backers** (consensus) improve the odds?

1.0.1 Method

For every roster wallet we pull **resolved** positions (`/closed-positions`), which tell us each bet's entry price (`avgPrice`), notional (`totalBought`), and outcome (`realizedPnl > 0` = it won). We group by (`market`, `outcome`), count how many sharps backed each, then simulate buying at `entry + slippage` and holding to resolution.

1.0.2 Read these caveats first — they decide how to interpret the numbers

1. **Survivorship bias (upward).** The roster was *selected* for being profitable, so a positive ROI is partly mechanical. The **informative** results are *relative*: does **backers 2** beat **backers 1**, and does win-rate exceed the price you pay?
2. **Entry-price optimism (upward).** We use the sharps' own entry price; in reality you follow later and pay more. We subtract a slippage haircut, but live will be worse.
3. **Hold-to-resolution only.** No mid-trade exits modelled here.

So treat a positive result as *necessary but not sufficient*. The decisive metric is **edge = hit_rate — avg_price_paid**: only if win-rate clears the price are you actually making money after buying in.

```
[1]: import importlib, pmc, json, time
importlib.reload(pmc)
from pmc import CFG, get_closed_positions
import pandas as pd, numpy as np

roster = json.load(open("roster.json"))["wallets"]
skill_by_wallet = {r["wallet"]: r.get("skill", 0.5) for r in roster}
print(f"{len(roster)} roster wallets")
```

27 roster wallets

1.1 1. Pull every roster sharp's resolved bets

One row per (wallet, resolved market-side): their entry price and whether it won.

```
[2]: rows = []
for i, r in enumerate(roster):
    w = r["wallet"]
    for p in get_closed_positions(w, max_positions=600):
        entry = float(p.get("avgPrice") or 0)
        rp = float(p.get("realizedPnl") or 0)
        cost = float(p.get("totalBought") or 0)
        if entry <= 0 or cost <= 0:
            continue
        rows.append({
            "wallet": w, "skill": skill_by_wallet.get(w, 0.5),
            "conditionId": p.get("conditionId"), "outcome": p.get("outcome"),
            "title": p.get("title"), "entry": entry,
            "realizedPnl": rp, "cost": cost,
            "won": 1 if rp > 0 else 0,
        })
    if (i+1) % 5 == 0:
        print(f"  pulled {i+1}/{len(roster)}")
bets = pd.DataFrame(rows)
print(f"{len(bets)} resolved sharp-bets across {bets['conditionId'].nunique()}
↳markets")
bets.head()
```

```
pulled 5/27
pulled 10/27
pulled 15/27
pulled 20/27
pulled 25/27
11495 resolved sharp-bets across 8937 markets
```

```
[2]:
```

	wallet	skill	\
0	0xd38b71f3e8ed1af71983e5c309eac3dfa9b35029	0.965	
1	0xd38b71f3e8ed1af71983e5c309eac3dfa9b35029	0.965	
2	0xd38b71f3e8ed1af71983e5c309eac3dfa9b35029	0.965	
3	0xd38b71f3e8ed1af71983e5c309eac3dfa9b35029	0.965	
4	0xd38b71f3e8ed1af71983e5c309eac3dfa9b35029	0.965	

	conditionId	\
0	0x43dc9ba9bb93ffb43c0ab680d405cc5c07e208fd2104...	
1	0xf01f219b5c84dedd842fc94649d82cd4c84f4c741093...	
2	0xcdedd4460eed2734e4680bf73b08872fe7347931814c...	
3	0xee09e7ad4876f3403678d8eb6de4898a780841de59e6...	
4	0xd054586fd01c9df8335e400e8f28aa903afa0875e940...	

	outcome	title	\
0	Connecticut Huskies	Spread: Illinois Fighting Illini (-1.5)	
1	Connecticut Huskies	Connecticut Huskies vs. Illinois Fighting Illini	

2	Connecticut Huskies	Michigan State Spartans vs. Connecticut Huskies
3	St. Johns Red Storm	Spread: Duke Blue Devils (-6.5)
4	Illinois Fighting Illini	Spread: Houston Cougars (-3.5)

	entry	realizedPnl	cost	won
0	0.480000	15600.000000	30000.00	1
1	0.460000	32400.000000	60000.00	1
2	0.570000	38700.000000	90000.00	1
3	0.508104	23264.993596	47296.57	1
4	0.560000	3566.120800	8104.82	1

1.2 2. Collapse to one row per market-side (the consensus unit)

backers = how many distinct sharps bet this exact side. **won** is the resolution (consistent across backers — same market, same outcome).

```
[3]: grp = []
for (cid, outcome), g in bets.groupby(["conditionId", "outcome"]):
    grp.append({
        "conditionId": cid, "outcome": outcome, "title": g["title"].iloc[0],
        "backers": g["wallet"].nunique(),
        "skill_sum": round(g["skill"].sum(), 3),
        "median_entry": round(g["entry"].median(), 4),
        "won": 1 if g["realizedPnl"].sum() > 0 else 0,
    })
mk = pd.DataFrame(grp)
print(f"{len(mk)} market-sides | backer distribution:")
print(mk["backers"].value_counts().sort_index().to_string())
```

10661 market-sides | backer distribution:

backers	
1	9929
2	648
3	69
4	12
5	3

1.3 3. Simulate the strategy at each consensus threshold

For **backers** $\geq N$, buy at **median_entry** + **slippage** (within the price band), hold to resolution. Equal stake per bet. The key columns: **hit_rate** (how often it won), **avg_price** (what you paid), **edge** ($\text{hit_rate} - \text{avg_price} \rightarrow \text{positive} = \text{real edge}$), and **roi_per_bet** (mean return per unit staked).

```
[4]: def simulate(df, n, slip=CFG.SLIPPAGE_TOLERANCE):
    s = df[df["backers"] >= n].copy()
    # only bets you could actually enter inside the price band
    s["buy"] = (s["median_entry"] + slip).clip(upper=0.99)
```

```

s = s[(s["buy"] >= CFG.PRICE_FLOOR) & (s["buy"] <= CFG.PRICE_CEILING)]
if s.empty:
    return None
# return per $1 staked on a binary bought at price p: win -> (1-p)/p, loss
-> -1
s["ret"] = np.where(s["won"] == 1, (1 - s["buy"]) / s["buy"], -1.0)
return {
    "min_backers": n,
    "n_bets": len(s),
    "hit_rate": round(s["won"].mean(), 3),
    "avg_price": round(s["buy"].mean(), 3),
    "edge": round(s["won"].mean() - s["buy"].mean(), 3),
    "roi_per_bet": round(s["ret"].mean(), 3),
}

results = pd.DataFrame([r for r in (simulate(mk, n) for n in [1, 2, 3, 4, 5])
if r])
print("Backtest summary (hold-to-resolution, equal stake):")
results

```

Backtest summary (hold-to-resolution, equal stake):

```

[4]:   min_backers  n_bets  hit_rate  avg_price  edge  roi_per_bet
0           1    9378    0.693    0.554  0.139    0.339
1           2     660    0.839    0.574  0.265    0.555
2           3      75    0.933    0.566  0.367    0.811
3           4      11    0.909    0.514  0.395    1.344
4           5       3    1.000    0.322  0.678    2.456

```

1.4 4. How to read this table

- **edge** > 0 is the headline: it means the bets won *more often than the price implied*, so copying them paid even after buying in. **edge** = 0 means no tradeable edge — the price already captured the sharps' information (exactly the latency story).
- Does **edge** / **roi_per_bet** rise as **min_backers** rises? If yes, **consensus adds value** — your original thesis holds. If it's flat or noisy (and **n_bets** collapses to a handful), consensus mostly just reduces sample size.
- **n_bets** at each level tells you the *frequency* you could actually trade.

Remember the upward biases (survivorship, entry-price). A *small or negative* edge here is a genuine red flag; a *large* edge is encouraging but should be discounted.

1.5 5. (Optional) Calibration — are higher-conviction bets better?

Bucket bets by entry price and compare actual win-rate to the price. Points above the diagonal = the sharps beat the market at that price.

```
[5]: base = mk[(mk["median_entry"] >= CFG.PRICE_FLOOR) & (mk["median_entry"] <= CFG.
    ↪PRICE_CEILING)].copy()
base["bucket"] = (base["median_entry"] * 10).round() / 10
cal = base.groupby("bucket").agg(n=("won", "size"), win_rate=("won", "mean")).
    ↪reset_index()
cal["win_rate"] = cal["win_rate"].round(3)
print("Calibration: price bucket vs actual win-rate (win_rate > bucket = edge)")
print(cal.to_string(index=False))
```

Calibration: price bucket vs actual win-rate (win_rate > bucket = edge)

bucket	n	win_rate
0.1	203	0.384
0.2	498	0.498
0.3	636	0.629
0.4	1302	0.679
0.5	3486	0.710
0.6	1528	0.736
0.7	847	0.802
0.8	721	0.818
0.9	296	0.821

1.6 What this tells us, and what's next

- If **edge is positive and grows with backers** → the strategy is real; we focus on the rare moments of genuine consensus and/or widen the roster so those moments appear more often, then paper-trade.
- If **edge is ~0 or negative** → copying *visible* positions is too late by construction. The honest pivots would be: (a) get faster/closer to their entries (real-time, not daily), (b) target a different, less-efficient corner of the market, or (c) conclude the public smart-money edge is already arbitrated away. Better to learn this now than with real money.

Either way, this is the experiment that should decide whether you fund the strategy. **Survivorship reminder:** for a bias-free read we'd eventually re-run this on a roster selected by data *before* the bet dates — happy to build that stricter version if the first pass looks promising.